



Core Java Fundamentals & OOPs

✉ rgupta.mtech@gmail.com

Learn Java
Build Your Future! 😊

Course Contents (Table of Contents)

“Same
Language
Infinite
Possibilities”

Write Once
Run Anywhere
!



Learn
Practice
Build
Grow

01		Fundamentals	11		Package Concept in Java
02		Java Basics	12		UML Basics
03		OOPs Basics	13		Inheritance in Java
04		OOPs Thinking Process	14		Overloading vs Overriding
05		Abstraction vs Encapsulation	15		Polymorphism
06		OO Core Concepts	16		Abstract classes
07		OO all	16		Final keyword
08		Interface Vs Abstract class	17		Interface Java
09		Basic OO syntax in Java	18		String in Java
10	this	this keyword in Java	19		String vs StringBuilder vs StringBuffer
			20		Wrapper classes in Java

Strong Basics → Clear Concepts → Better Programmers

Let's Make Java Simple! 😊



Java – OOPL + JVM + Libraries

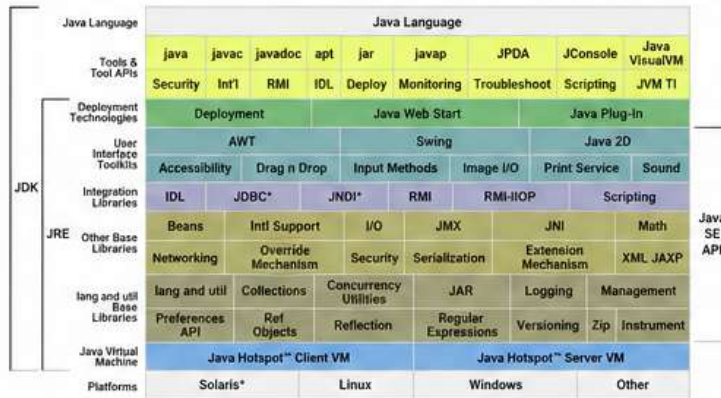
Write Once → Run Anywhere

Simple
Powerful
Everywhere

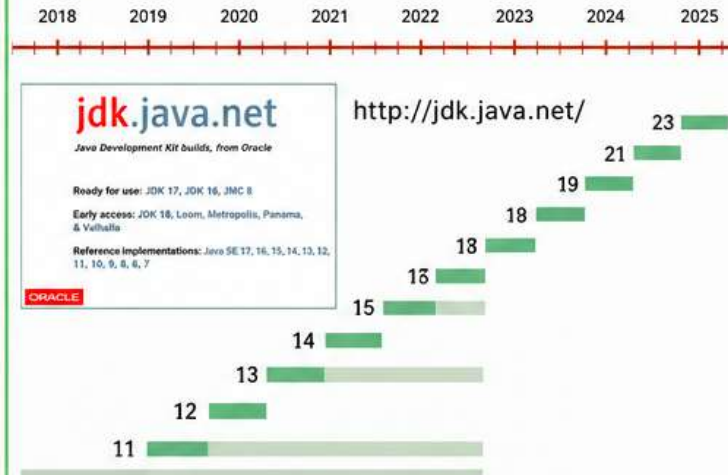
1 What is Java?

Java = OOPL + JVM + lib

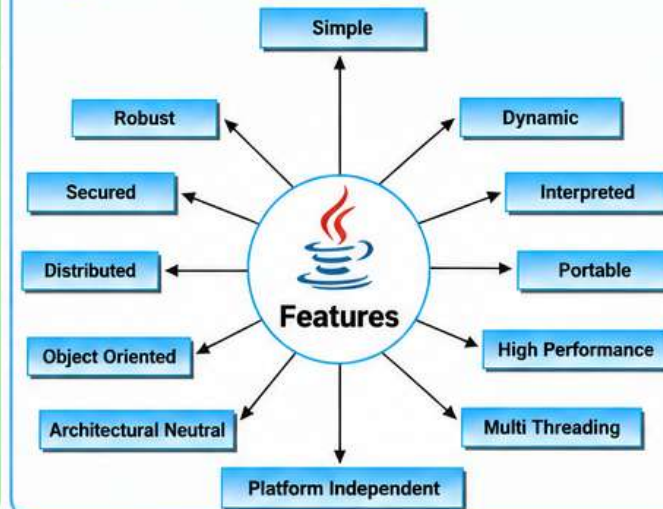
□ How Java is different then C++?



2 Java Versions

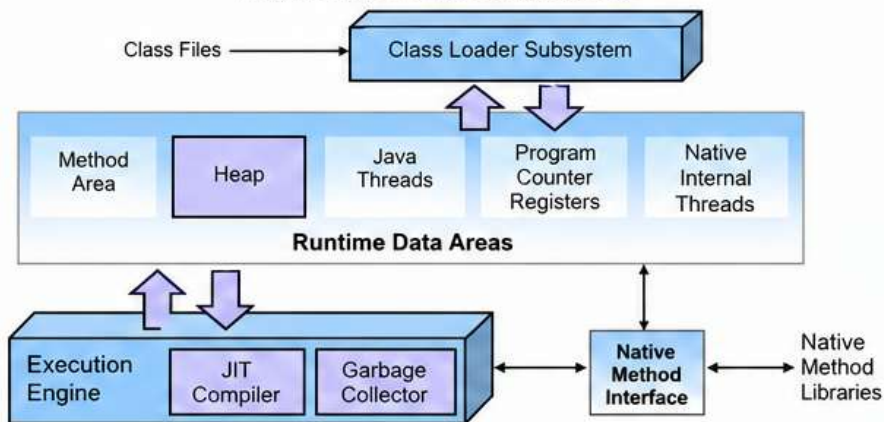


3 How Sun define Java?



4 Understanding JVM

Key HotSpot JVM Components



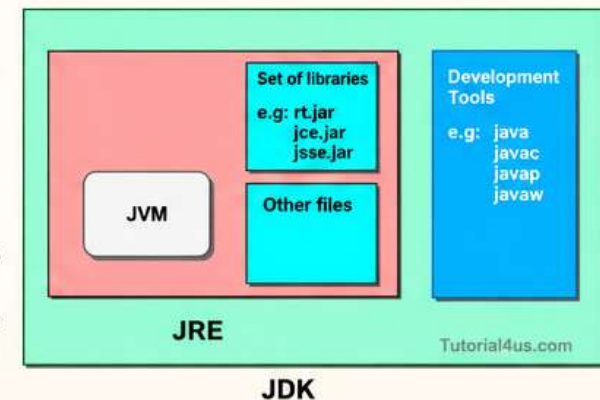
5 Some common buzzwords

JVM

- Java Virtual Machine JVM
- Pick byte code and execute on client machine Platform dependent...

JDK

- Java development kit, developer need it
- Byte Code
- Highly optimize instruction set, designed to run on JVM Platform independent
- Java Runtime environment, Create an instance JVM, must be there on deployment machine.
- Platform dependent



Java = A robust, secure, object-oriented, platform-independent language powered by the JVM and a rich set of libraries.

More than a Language...
A Platform for the World!



Java Basics

Learn → Code → Practice → Build

Same Language
More Possibilities

Write Once
Run Anywhere

1 Lab setup and Hello world

Lab set-up

- Java 21
- IntelliJ IDEA Community Edition



Maven

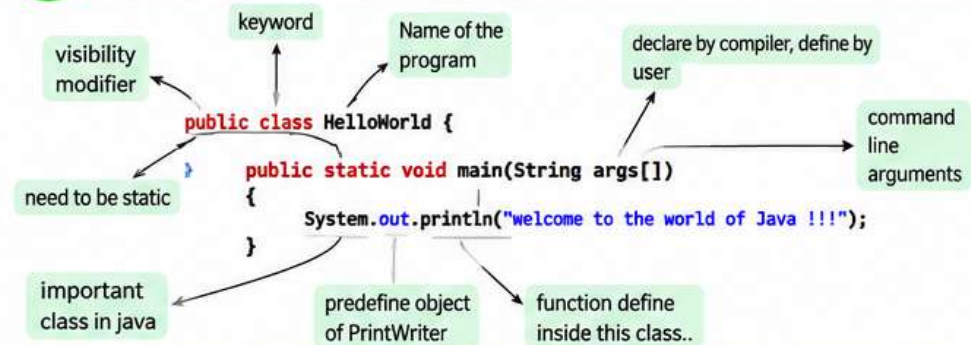
```

Project
├── src
│   └── is
│       ├── HelloWorld
│       └── HelloWorld.java
├── External Libraries
└── Scratches and Consoles

HelloWorld.java
1 public class HelloWorld {
2     public static void main(String[] args) {
3         System.out.println("welcome to the
4             world of Java !!!");
5     }
6 }
7
8
9

```

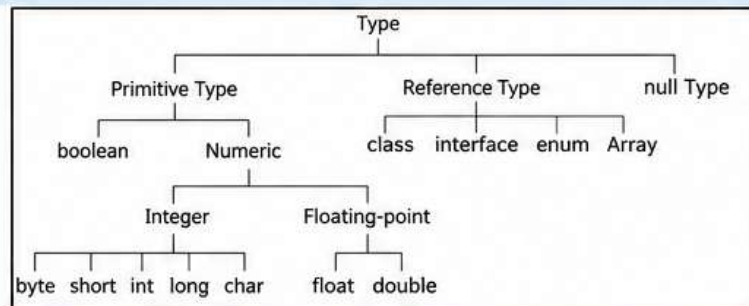
2 Analysis of Hello World



3 Java Data Type

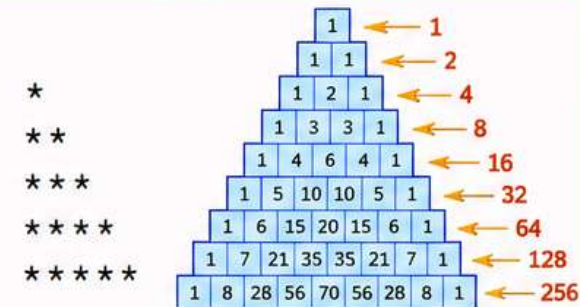
2 types

- Primitive data type
- Reference data type



4 Procedural Programming

- if..else
- switch
- Looping; for, while, do..while as usual in Java as in C/C++
- Don't mug the program/logic
- Follow dry run approach
- Try some programmes:
 - Create
 - Factorial program
 - Prime No check
 - Date calculation

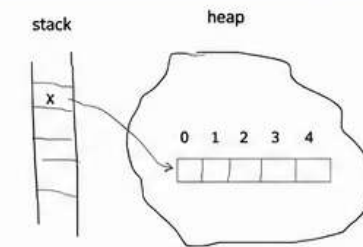


5 Array are object in java

How Java array different from C/C++ array?

One Dimensional array

Initialization	<code>int a[] = new int [12];</code>											
Value	1	2	3	4	6	7	8	9	10	11	12	
Index	a[0]	a[1]	a[2]	a[4]	a[5]	a[6]	a[7]	a[8]	a[9]	a[10]	a[11]	
System.out.print(a[5]);	Output: 6											



Java Array: its different then C/C++

★ Key Takeaways

- ✓ Set up Java 21 and IntelliJ IDEA (Community Edition)
- ✓ Understand the structure of a simple Java program
- ✓ Know Java data types (primitive and reference)
- ✓ Use control statements and loops for problem solving
- ✓ Array in Java is an object and different from C/C++
- ✓ Practice with small programs (factorial, prime, date etc.)

Small Programs
Build Big Skills! 😊

Keep Coding
Keep Exploring
With Java ☕



OO Basics

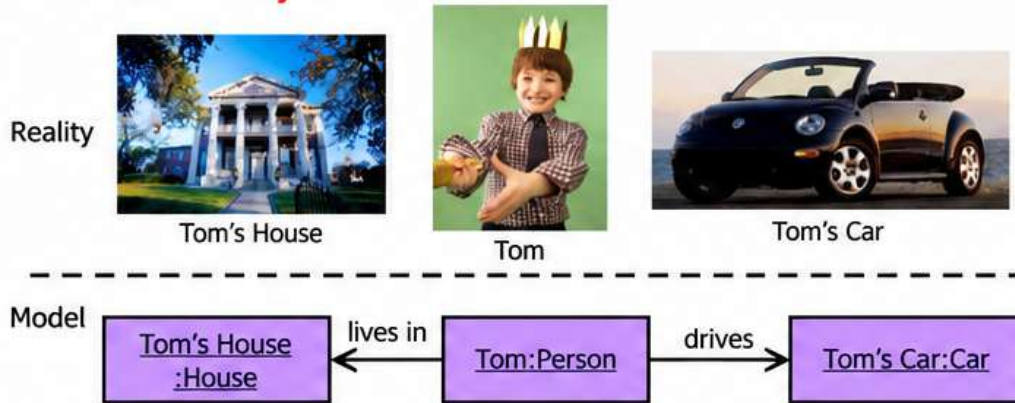
Real World → Objects → Classes → Better Software

Think in Objects
Build Real Solutions

Objects
Model
Real World
Things! 😊

1 What is Object Orientation?

- OO is a way of looking at a software system as a collection of **interactive objects**



2 What is Object?

Informally, an object represents an entity, either physical, conceptual, or software.

Physical entity



Tom's Car

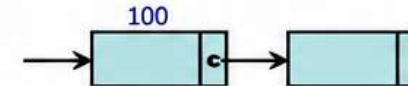
Conceptual entity



US\$100,000,000,000

Bill Gate's bank account

Software entity



Object can be:

- Physical thing (e.g. car, person)
- Conceptual thing (e.g. bank account, employee)
- Software thing (e.g. data structure, object in program)

3 What is class?

- A class is the blueprint from which individual objects are created.
- An object is an instance of a class.



Object factory



Cookie Cutter

```

public class StudentTest {
    public static void main(String[] args) {
        Student s1 = new Student();
        Student s2 = new Student();
    }
}
  
```

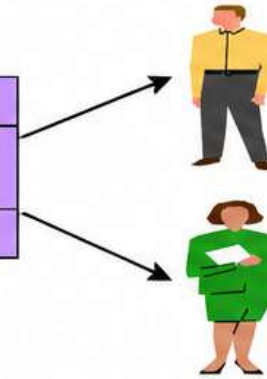
- class -

```

public class Student {
    private String name;
    // ...
}
  
```

4 class and objects

- All the objects share the same attribute names and methods with other objects of the same class
- Each object has its own value for each of the attribute



harry:Person	
name	Harry
dateOfBirth	8/12/1975
+ getAge()	

mary:Person	
name	Mary
dateOfBirth	8/12/1980
+ getAge()	

Objects model real world entities. Classes provide the blueprint. Together they help us design better, maintainable and reusable software.

Same Concepts...
Bigger Possibilities with Java!



OO Basics

Program Smart
Solve Real Problems

From Thinking → Modeling → Coding → Real World Solutions

1 Programmer are Not typist but thinker

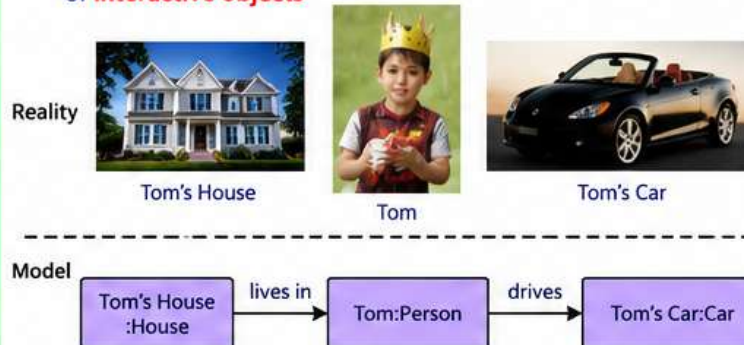
Programmer are Not typist but thinker



Don't just write code,
Think, Analyze, Design and Solve Problems!

2 What is Object Orientation?

- OO is a way of looking at a software system as a collection of **interactive objects**



3 What is Object?

- Informally, an object represents an entity, either physical, conceptual, or software.

Physical entity



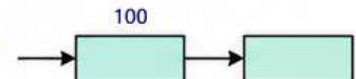
Tom's Car

Conceptual entity



US\$100,000,000,000

Software entity



4 What is class?

- A class is the **blueprint** from which individual objects are created.
- An object is an instance of a class.



```

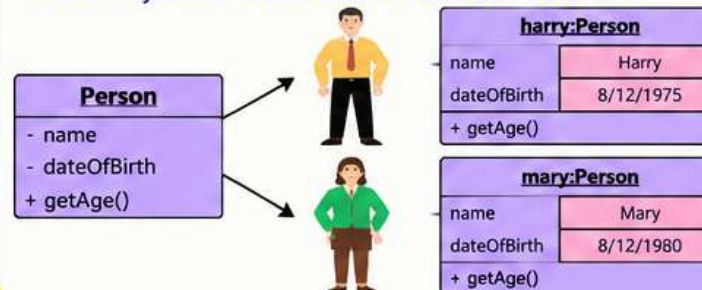
public class StudentTest {
    public static void main(String[] args) {
        Student s1 = new Student();
        Student s2 = new Student();
    }
}
  
```

```

- class -
public class Student {
    private String name;
    // ..
}
  
```

5 class and objects

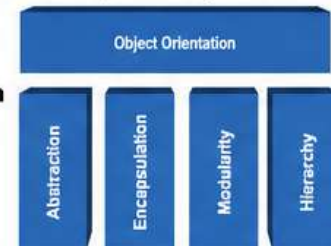
- All the objects share the same attribute names and methods with other objects of the same class
- Each object has its own value for each of the attribute



6 Pillars of object oriented programming

Basic Principles of Object Orientation

- Abstraction**
- Encapsulation**
- Modularity**
- Hierarchy**



7 Learning OOPs Basics



8 Beyond Basics – Important Concepts

- Objects and Classes
- Attributes and Methods
- Constructors
- Access Modifiers
- This keyword
- Static members
- Inheritance (IS-A)
- Polymorphism (Overloading & Overriding)
- Abstraction (Interfaces & Abstract classes)
- Encapsulation (Data hiding)
- Composition (HAS-A)
- Method Chaining
- Immutability
- Object Lifecycle (Creation, Usage, GC)
- Real-world Modeling (e.g., Bank, E-commerce)
- Best Practices and Clean Code

9 Key Takeaways

- Think in terms of Objects
- Model real-world entities
- Use classes to create objects
- Apply OOP principles in design
- Start simple, practice, build projects
- OOP skills are essential for frameworks and real-world development

Think
Model
Design
Code
Build
Better Software

Objects today,
Better software tomorrow!

Good Software Starts with Good Thinking ... Master OOP, Build Real Solutions!

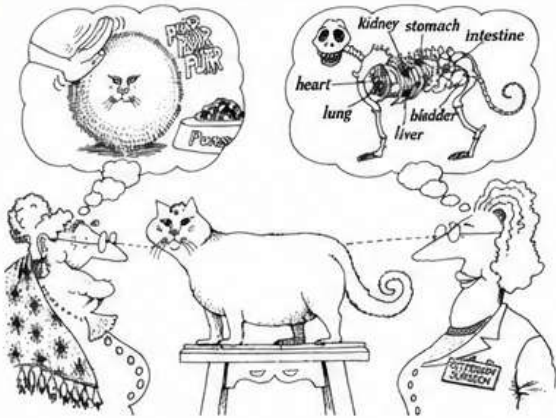


Abstraction vs Encapsulation

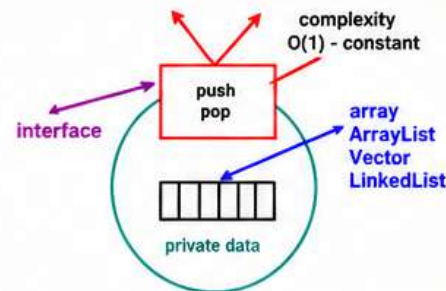
Hide Complexity → Expose Only What is Needed

Two Concepts
One Better Design!

1 Abstraction



Abstraction focuses upon the essential characteristics of some object, relative to the perspective of the viewer



Abstraction is the Design principle of separating interface (not java keyword) from implementation so that the client only concern with the interface

In Java, abstraction can be achieved using an interface or abstract class.

2 Encapsulation

How to implement Encapsulation provide private visibility to an instance variable

```
class Account{  
    private int account_number;  
    private int account_balance;  
  
    public void show Data(){  
        // code to show data  
    }  
  
    public void deposit(int a){
```

Encapsulation the process of restructuring access to inner implementation details of a class. Internal not exposed to the outside world For example, sealed mobile, if you open it guarantee is violated



How to implement Encapsulation provide private visibility to an instance variable

You can implement encapsulation using Access Modifiers (Public, Protected & Private).

3 Abstraction vs Encapsulation

Abstraction

General concept of extracting common features and hiding unnecessary details.

Use interface or abstract class.

Solves the problem at design level.

Means hiding implementation using abstract class and interface.

Encapsulation

Mechanism that binds together code and the data it manipulates, and keeps both safe from outside interference and misuse.

Use access modifiers (public, protected & private).

Solves the problem at implementation level.

Means hiding data using private variables (and getters/setters).

Both are complementary concepts.
Use abstraction to show only what is needed and encapsulation to protect how it works!

Abstraction shows the important things. Encapsulation protects the details. Together they make better, maintainable and real-world software.



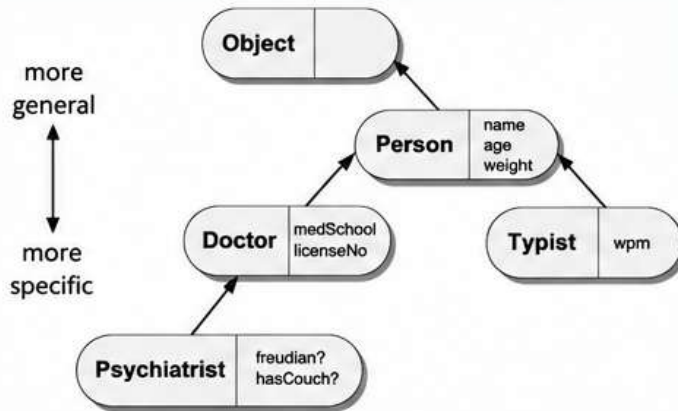
OOP Core Concepts

Model Real-World Relationships → Build Flexible & Maintainable Code

Re-use
Extend
Be Flexible
Build Better! 😊

1 Inheritance

Hierarchy give rise to inheritance and polymorphism



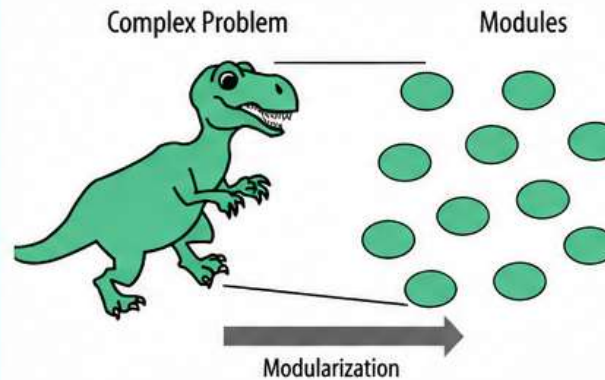
When a class derives from another class, the child class inherits all the public and protected properties and methods from the parent class (extends keyword).

Key Points

- ✓ Creates a **parent-child** relationship (IS-A relationship).
- ✓ Promotes **code reusability**.
- ✓ Forms a **hierarchy** of classes.
- ✓ Child class can add its own properties and methods.
- ✓ Supports single, multilevel, hierarchical and multiple inheritance (via interfaces in Java).

2 Modularity

Break a complex problem into smaller, manageable modules.



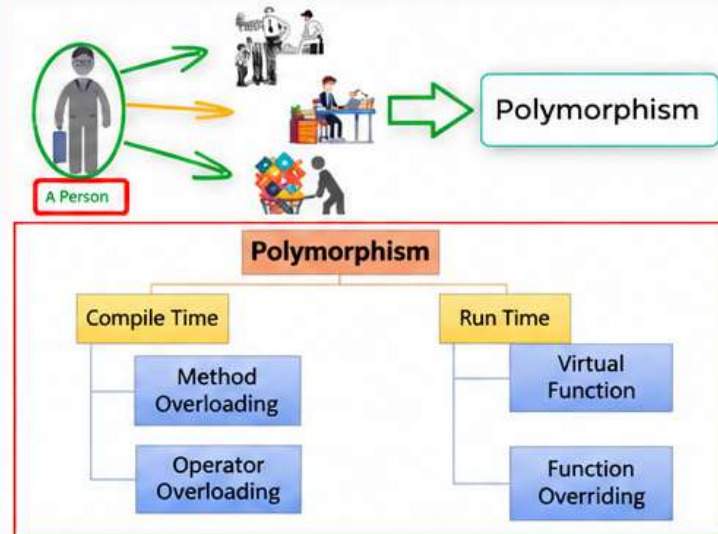
Modularity helps in breaking a complex problem into smaller, manageable modules. Each module has a specific responsibility and can be developed, tested and maintained independently.

Key Points

- ✓ Split a large system into smaller modules/classes.
- ✓ Each module has a clear responsibility.
- ✓ Improves **maintainability** and readability.
- ✓ Modules can be developed and tested separately.
- ✓ Makes the system easier to extend and reuse.

3 Polymorphism

Same interface, different behavior.



Polymorphism is one of the core concepts of OOP and describes situations in which something occurs in several different forms. In Java, you can access objects of different types through the same interface.

Key Points

- ✓ Same interface, different implementations.
- ✓ Achieved using **method overloading** (compile time).
- ✓ Achieved using **method overriding** (run time).
- ✓ Provides **flexibility** and **extensibility**.
- ✓ Common interfaces allow uniform treatment of different objects.

Use Inheritance to Reuse, Modularity to Organize, and Polymorphism to Stay Flexible.

Good Design
Lasts Longer! 😊



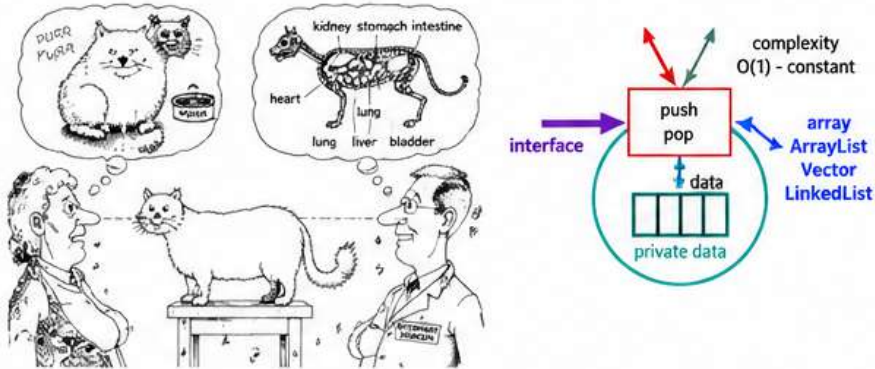
OOP Pillars in Action

Key Concepts → Clean Design → Reusable Code → Real World Applications

Same Concepts
Bigger Possibilities!

1 Abstraction

Abstraction focuses upon the essential characteristics of some object, relative to the perspective of the viewer.



Abstraction is the design principle of separating interface (not java keyword) from implementation.

In Java, abstraction can be achieved using an interface or abstract class.

2 Encapsulation

Encapsulation is the process of restructuring access to inner implementation details of a class. Internal not exposed to the outside world.

How to implement Encapsulation provide private visibility to an instance variable

```

class Account{
    private int account_number;
    private int account_balance;

    public void show Data(){
        // code to show data
    }

    public void deposit(int a){

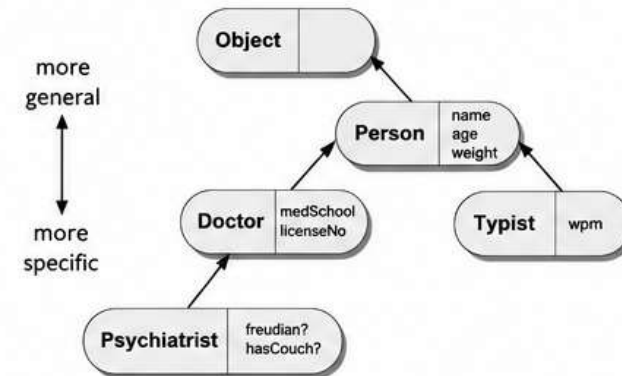
```

Class Data Methods

Use access modifiers (public, protected & private) to hide data (use getters and setters if needed).

3 Inheritance

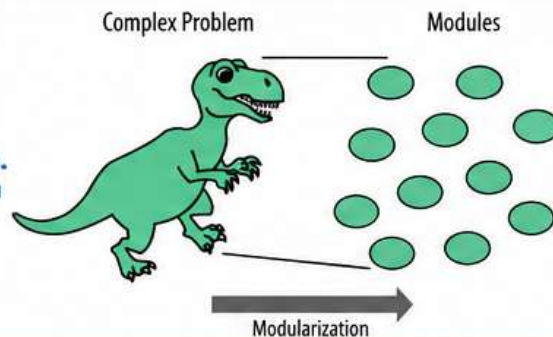
Hierarchy give rise to inheritance and polymorphism



When a class derives from another class, the child class inherits all the public and protected properties and methods from the parent class (extends keyword).

4 Modularity

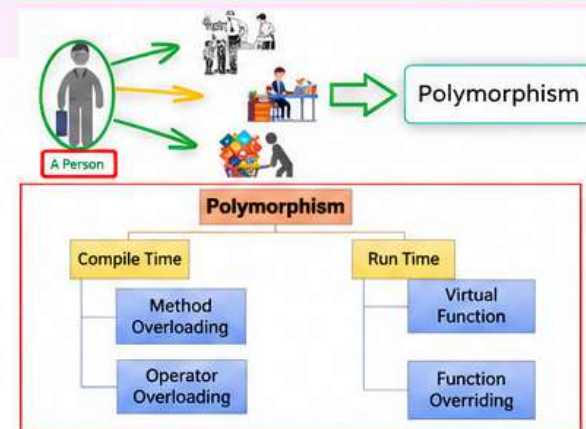
When a class derives from another class. The child class will inherit all the public and protected properties and methods from the parent class. In addition, it can have its own properties and methods. An inherited class is defined by using the extends keyword



Modularity helps in breaking a complex problem into smaller, manageable modules.

5 Polymorphism

Polymorphism is one of the core concepts of object-oriented programming (OOP) and describes situations in which something occurs in several different forms. In computer science, it describes the concept that you can access objects of different types through the same interface.



Abstraction. Encapsulation. Inheritance. Modularity. Polymorphism.
Together they make your code simple, flexible, reusable and ready for the real world.

Think in Objects
Build Better Software! 😊



Interface vs Abstract class

Two Ways to Define a Blueprint

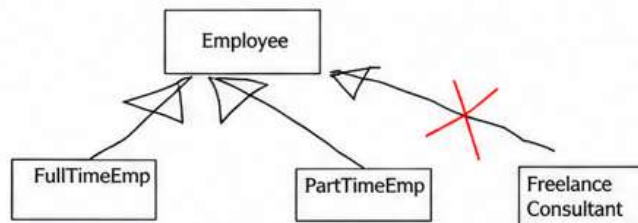
Same Goal...
Different Purpose!

1 What is Interface?

Interface: Specification of a behavior
The interface represents some features of a class

What an object can do?

Standardization of a behavior



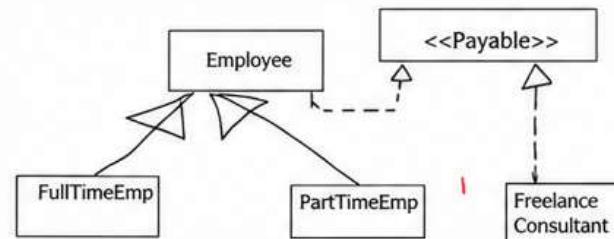
When we should go for interface and when we should go for abstract class?
Interface break the hierarchy

2 What is Abstract class?

Abstract class: generalization of a behavior
The incomplete class that required further specialization

What an object is?

IS-A SavingAccount IS-A Account

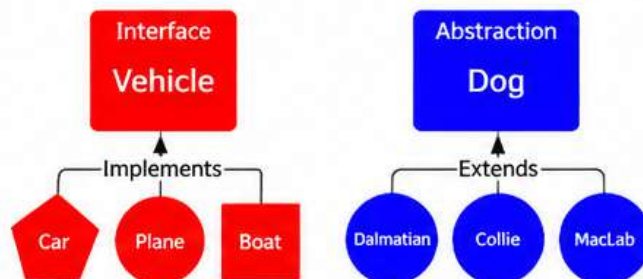


3 Feature Comparison

	Interface	Abstract Class
Constructors	✗	✓
Static Fields	✓	✓
Non-static Fields	✗	✓
Final Fields	✓	✓
Non-final Fields	✗	✓
Private Fields & Methods	✗	✓
Protected Fields & Methods	✗	✓
Public Fields & Methods	✓	✓
Abstract methods	✓	✓
Static Methods	✓	✓
Final Methods	✗	✓
Non-final Methods	✓	✓
Default Methods	✓	✗

4 Interfaces vs. Abstract Classes

Interfaces vs. Abstract Classes



5 When to use Interface?

- When we should go for interface and when we should go for abstract class?
- Use interface when you want to break the hierarchy.
- Use interface to standardize a behavior.
- Use interface when different classes need to do the same thing, but are not related by inheritance.
- Interface represents some features of a class.

6 When to use Abstract class?

- Use abstract class when you need a generalization of a behavior.
- Use abstract class when you want to provide common implementation.
- Use abstract class when classes are related (IS-A relationship).
- Use abstract class when the class is incomplete and requires further specialization.
- Abstract class can have fields, constructors and both abstract and concrete methods.

Choose the right tool for the right purpose – Interfaces define what to do, Abstract classes provide a base to build upon.

Design Smart
Code Better! 😊



Basics of OOP Syntax in Java

Define Class → Create Objects → Use Constructors → Access Members

Think in Objects
Write Better Code!

1 What can go inside a class?

```

public class A {
    int i;          // instance variable

    static int j;   // static variable

    //method in class
    public void foo(){
        int i;     //local variable
    }

    public A(){     //default constructor

    public A(int j) //parameterized ctr
    {
        .....;
    }

    //getter and setter
    public int getI(){return i;}
    public void setI(int i){this.i=i;}
}

```

A class can contain

- Instance variables
- Static variables
- Methods
- Constructors
- Local variables (inside methods)
- Getters and setters
- Different access modifiers
- Any valid Java code

2 Creating Classes and Object

```

class Account{
    public int id;
    public double balance;
    //.....
}

public class AccountDemo{
    public static void main(String[] args) {
        Account ac=new Account();
        ac.id=22;
    }
}

```

→ killing encapsulation

- A class is a blueprint.
- An object is an instance of a class.
- Using public fields directly is not recommended (breaks encapsulation).

3 Correct way (Encapsulation)

```

class Account{
    private int id;
    private double balance;

    public int getId() {
        return id;
    }
    public void setId(int id) {
        this.id = id;
    }
    public double getBalance() {
        return balance;
    }
    public void setBalance(double balance) {
        this.balance = balance;
    }
}

public class AccountDemo{
    public static void main(String[] args) {
        Account ac=new Account();
        //ac.id=22; will not work
        ac.setBalance(2000); //correct way
    }
}

```

Use private fields with getters and setters

- Hide data from outside world.
- Control access through methods.
- Keeps object in a valid state.
- This is the correct way to implement encapsulation.

4 Java Constructor

- Initialize state of the object
- Special method have same name as that of class
- Can't return anything
- Can only be called once for an object
- Can be private
- Can't be static*
- Can be overloaded but can't be overridden*
- Three type of constructors
 - Default
 - Parameterized
 - Copy constructor

```

class Account{
    private int id;
    private double balance;

    //default ctr
    public Account() {
        //.....
    }

    //parameterized ctr
    public Account(int i, double b) {
        this.id=i;
        this.balance=b;
    }

    //copy ctr
    public Account(Account ac) {
        //.....
    }
}

```

5 Types of Variables

Instance Variable

```
int i;
```

Belongs to object.
Each object has its own copy.

Static Variable

```
static int j;
```

Belongs to class.
Shared by all objects.

Local Variable

```

public void foo(){
    int i;
    // local variable
}

```

Declared inside a method.
Scope is within the method only.

6 Key Takeaways

- ✓ Class defines the structure (fields, methods, constructors).
- ✓ Create objects using new keyword.
- ✓ Use private fields with getters and setters.
- ✓ Use constructors to initialize object state.
- ✓ Understand instance, static and local variables.
- ✓ Encapsulation helps protect the data.
- ✓ Use proper access modifiers.
- ✓ This forms the foundation for Inheritance, Polymorphism and other OOP concepts.

Small
Classes
Better
Software!
😊

Objects are the building blocks. Write classes that model real-world entities.



The **this** Keyword in Java

Refers to the current object

*Same object,
Clearer code!*

1 Need of "this" ?

```
class Account{
    private int id;
    private double balance;
```

```
//parameterized ctr
public Account(int id, double balance) {
    id=id;
    balance=balance;
}
```

Which id assigned to which id ?

- Constructor parameters have the same names as instance variables.
- The compiler gets confused which variable to use.

2 Using "this" to resolve confusion

```
class Account{
    private int id;
    private double balance;

    //parameterized ctr
    public Account(int id, double balance) {
        this.id = id;
        this.balance = balance;
    }
```

"this" refers to the current object's instance variable

- **this.id** refers to the instance variable of the current object.
- The right side (**id, balance**) refers to the constructor parameters (local variables).
- **this** helps differentiate between them.

3 Calling one constructor from another

```
class Account{
    private int id;
    private double balance;
```

```
//default ctr
public Account() {
    this(22,555.0);
}
```

Calls another constructor in the same class

```
//parameterized ctr
public Account(int id, double balance) {
    this.id=id;
    this.balance=balance;
}
```

- **this(...)** is used to call one constructor from another constructor (constructor chaining).
- It must be the first statement in the constructor.
- Helps in reusing initialization code.

4 Complete Example

```
class Account{
    private int id;
    private double balance;

    //default ctr
    public Account() {
        this(22,555.0);
    }

    //parameterized ctr
    public Account(int id, double balance) {
        this.id=id;
        this.balance=balance;
    }

    //copy ctr
    public Account(Account ac) {
        //.....
    }
```

Instance variables (private)

Calling another constructor

Using this to refer to instance variables

Copy constructor (example)

5 What is "this" ?

- "this" is a reference to the current object.
- Used to differentiate instance variables from local variables (like method parameters).
- Used to call one constructor from another (constructor chaining).
- Can be used to refer to the current object anywhere in the class.

6 Key Points

- **this.id** → instance variable
- **id** → constructor parameter (local variable)
- **this(...)** → calls another constructor
- Must be the first statement in a constructor
- Helps avoid confusion and makes code clearer

7 Java Constructor (Quick Recap)

- Initialize state of the object
- Same name as the class
- Can't return anything
- Can only be called once for an object
- Can be private
- Can't be static
- Can be overloaded but can't overridden
- Three types of constructors:
 - Default
 - Parameterized
 - Copy constructor

this makes constructors more powerful and clear!

Understand "this" today, write better object-oriented code tomorrow!



Java

Objects create opportunities!



Package Concept in Java

Group Related Classes Like Containers (Jars Inside Jars)

Organize
Reuse
Write Better Code!

Packages
Keep Your Code
Clean & Manageable! 😊

1 What is a Package?

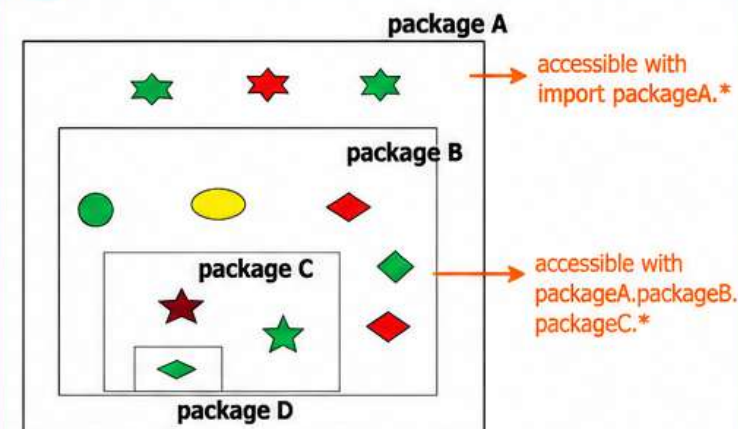
- Packages are Java's way of grouping a number of related classes and/or interfaces together into a single unit.
- Packages act as "containers" for classes.

"Just like a jar keeps toffees together, a package keeps related classes together."

2 Jar Analogy (Nested Packages)

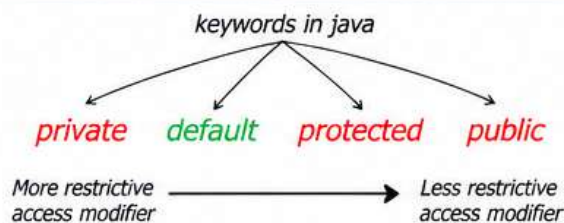


3 Package Structure (Diagram)



4 Visibility Modifiers

- For instance variables and methods
 - public
 - protected
 - default (package level)
 - private
- For classes
 - public and default



5 Examples

1. Default visibility (same package only)

```
pack packA;  
  
class A{  
    public void foo(){  
    }  
}
```

Class A has default visibility, hence can be accessed in the same package only.

2. Access from another package (Not allowed)

```
pack packB;  
import packA.*;  
class B{  
    public void boo(){  
        A a=new A(); // Not allowed  
    }  
}
```

Class A is not public, so it cannot be accessed from another package.

3. Make class public (Allowed)

```
pack packA;  
  
public class A{  
    protected void foo(){  
    }  
}
```

Now class A is public and can be accessed from other packages.

4. Access protected member in subclass

```
pack packB;  
import packA.*;  
class C extends A{  
    public void foo2(){  
        foo(); // allowed  
    }  
}
```

Protected data can be accessed in the same package and all subclasses in other packages (provided the class itself is public).

6 Key Points

- ✓ Packages group related classes and interfaces.
- ✓ Packages act as containers.
- ✓ A package can contain sub-packages.
- ✓ Use import to access classes from another package.
- ✓ Visibility modifiers control access to classes and members.
- ✓ Default (no modifier) is accessible within the same package.
- ✓ Protected is accessible in the same package and subclasses (in other packages, if the class is public).
- ✓ Classes can be public or default (package level).

Real-World Analogy

- Big jar → package A (e.g., com)
- Medium jar → package B (e.g., com.bank)
- Small jar → package C (e.g., com.bank.service)
- Inner jar → package D (e.g., com.bank.service.impl)
- You can take toffees (use classes) only from the jar you open (or jars you import).

Better
Organization
Bigger Possibilities!

Packages bring structure to your code, just like jars keep things organized!

Organized Code
Happy Developer! 😊

UML Basics and Relationships Between Objects

Visualize → Design → Communicate → Build Better Software

Same ideas
Clearer design
Better software! 😊

1 What is UML?

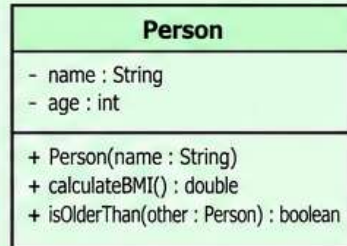
- UML (Unified Modeling Language) is a standard way to visualize, design and document software systems.
- UML 2.0 has 12 types of diagrams.
- Most important: Class diagram, Use case diagram and Sequence diagram.
- Helps us understand the system before coding.
- You should know how to read UML diagrams correctly.

"A picture is worth a thousand lines of code
— UML shows the big picture!"

2 Class Diagram (Structure of a Class)

Shows classes, their attributes and methods.

UML Notation



Equivalent Java Code

```
public class Person {
    private String name;
    private int age;

    public Person(String name) { ... }

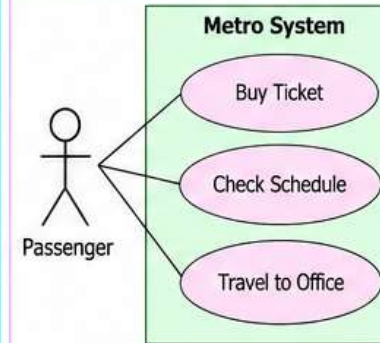
    public double calculateBMI() {...}

    public boolean isOlderThan(Person other) { ... }
}
```

+ Public - Private # Protected ~ Package(default)

3 Use Case Diagram (What the system does)

Shows the interaction between users (actors) and the system.

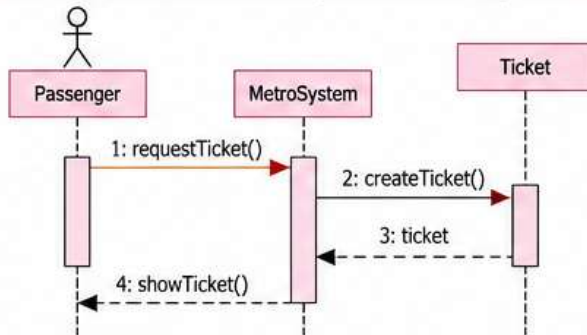


Key Points

- Actor:** a user of the system (e.g., Passenger)
- Use case:** a service provided by the system (e.g., Buy Ticket)
- Shows "what" the system does, not "how".

4 Sequence Diagram (How it works)

Shows the order of messages between objects.



Key Points

- Shows the time order (sequence) of interactions.
- Objects are shown with lifelines.
- Messages are sent from one object to another.
- Useful to understand the flow of a scenario.

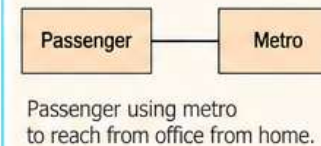
5 Relationships Between Objects

Shows how classes/objects are related to each other.

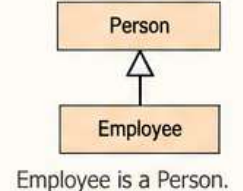
Relationship	UML Notation	Example	Meaning
Association (Use-a)	—	Passenger — Metro	One object uses another.
Inheritance (Is-a)	— >	Employee — > Person	Employee is a Person.
Composition (Strong Has-a)	—◆	Apartment ◆ Flat	Flat is part of Apartment (strong relation).
Aggregation (Weak Has-a)	—◇	Musician ◇ MusicGroup	Musician is with MusicGroup (weak relation).
Dependency	-.->	Employee -.-> Report	Uses temporarily (e.g., as a parameter).

6 UML Relationship Examples

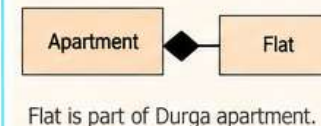
Association (Use-a)



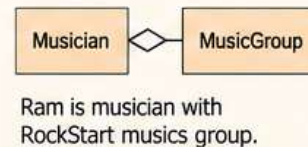
Inheritance (Is-a)



Composition (Has-a)



Aggregation (Has-a)



UML helps you think, design and communicate better object-oriented solutions.

Plan Better
Build Smarter! 😊

Inheritance in Java

Acquire → Reuse → Extend → Build Better Code

One class,
Many possibilities!

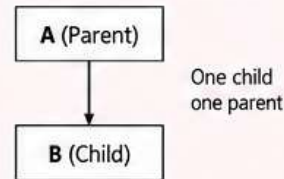
1 What is Inheritance?

- Inheritance is the inclusion of behaviour (i.e. methods) and state (i.e. variables) of a base class in a derived class so that they are accessible in the derived class.
- It provides code reusability.
- The class whose members are inherited is called the Super class (Parent).
- The class that inherits the members is called the Sub class (Child).

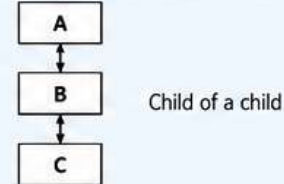
"Child class gets the properties of parent class and can also have its own properties."

2 Types of Inheritance (in Java)

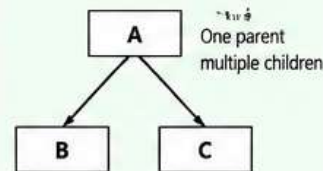
Single Inheritance



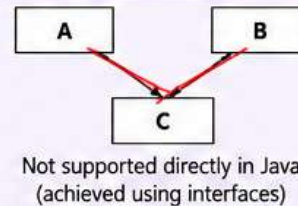
Multilevel Inheritance



Hierarchical Inheritance



Multiple Inheritance



3 Inheritance Example in Java

```
class A {  
    int i;  
    A() { System.out.println("Default ctr of A"); }  
    A(int i) { System.out.println("Parameterized ctr of A"); }  
}  
  
class B extends A {  
    int j;  
    B() {  
        super(); // calls parent constructor  
        System.out.println("Default ctr of B");  
    }  
    B(int i, int j) {  
        super(i); // pass value to parent constructor  
        this.j = j;  
        System.out.println("Parameterized ctr of B");  
    }  
}
```

Key Keywords

- **extends** : used for inheritance
- **super** : used to access parent class constructors and members

Points to Note

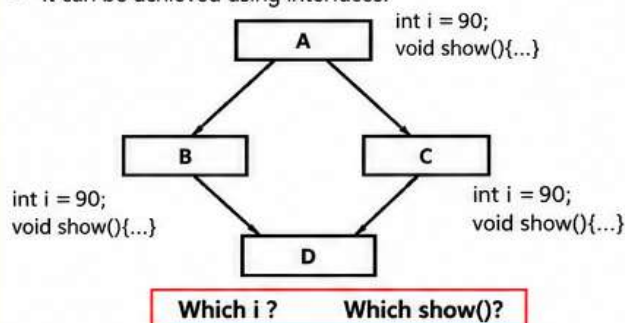
- Child class inherits variables and methods of parent class.
- Use **super** to call parent constructor.
- Child class can have its own members.
- Constructors are not inherited, but can be invoked using **super**.

4 Diamond Problem

- In multiple inheritance, a class can inherit the same method from two different paths, which creates ambiguity.
- Java does not support multiple inheritance with classes to avoid this problem.
- It can be achieved using interfaces.



Diamond Problem



5 Superclass vs Subclass

Feature	Superclass (Parent)	Subclass (Child)
Role	Base class	Derived class
Members	Declared and defined here	Inherits parent's members and can have its own
Keyword	-	extends
Access	Can be accessed from child (as per visibility)	Can access parent members (as per visibility)
Constructor	Called automatically (or using super)	Can call parent constructor using super()
Example	class A { ... }	class B extends A { ... }

6 Key Takeaways

- ✓ Inheritance promotes code reusability.
- ✓ Superclass (Parent) and Subclass (Child) concept.
- ✓ Use **extends** keyword.
- ✓ Use **super** to call parent constructor or access parent members.
- ✓ Multiple inheritance with classes is not supported due to Diamond problem (use interfaces).
- ✓ Helps in building a clear and maintainable class hierarchy.
- ✓ Constructors are not inherited, but can be invoked using **super()**.

Reuse what works,
Build what's next! 😊

Inheritance: Inherit. Extend. Innovate.



Method Overloading vs Overriding in Java

Same Name, Different Purpose!

Understand
Use Correctly
Write Better Code!

1 Method Overloading

- Overloading means having multiple methods in the same class with the same name but different method signatures (different number or type of parameters).
- It allows you to define the same operation in different ways for different input.
- Return type can be same or different, but parameter list must be different.
- Constructors can also be overloaded.

"Same method name, different parameters
— compile time polymorphism"

Example: Method Overloading

```
class MyClass {  
    public void getInvestAmount(int rate) {  
        System.out.println("Rate: " + rate);  
    }  
  
    public void getInvestAmount(int rate, long principal) {  
        System.out.println("Rate: " + rate + ", Principal: " + principal);  
    }  
  
    public void getInvestAmount(double amount) {  
        System.out.println("Amount: " + amount);  
    }  
}
```

Constructor Overloading

```
class Account {  
    int id;  
    double balance;  
  
    public Account() { // default constructor  
        this(0,0.0);  
    }  
  
    public Account(int id) { // 1 parameters  
        this(id,0.0);  
    }  
  
    public Account(int id, double balance) { // 2 parameters  
        this.id = id;  
        this.balance = balance;  
    }  
}
```

Constructors can be overloaded to create objects in different ways.

2 Method Overriding

- Overriding means having the same method (name and signature) in the parent class and providing a new implementation in the child class.
- It allows the subclass to change or extend the behavior of the parent class method.
- Return type must be the same (or covariant).
- Cannot reduce the visibility (e.g., public to private).
- It is achieved using inheritance.

"Same method name and signature, different implementation
— run time polymorphism"

Example: Method Overriding

```
class BaseClass {  
    public void getInvestAmount(int rate) {  
        System.out.println("BaseClass: Rate = " + rate);  
    }  
}  
  
class MyClass extends BaseClass {  
    @Override  
    public void getInvestAmount(int rate) {  
        System.out.println("MyClass: Rate = " + rate);  
    }  
}
```

Subclass provides its own implementation of the parent method (same signature).

Run Time Behavior

```
public class Test {  
    public static void main(String[] args) {  
        BaseClass obj = new MyClass();  
        obj.getInvestAmount(5); // Calls MyClass method (overridden)  
    }  
}
```

Even though the reference is of **BaseClass** type, the overridden method in **MyClass** is called at runtime.

3 Key Differences

Feature	Overloading	Overriding
Where	Same class	Parent and Child class
Method name	Same	Same
Parameters	Must be different (number/type/order)	Same
Return type	Can be same or different	Must be same (or covariant)
Access modifier	Any (no restriction)	Cannot reduce visibility (e.g., public → private)
Inheritance required	Not required	Required
Polymorphism	Compile time (static)	Run time (dynamic)
Purpose	Same operation, different inputs	Change/extend behavior in subclass
Example	getInvestAmount(int), getInvestAmount(double)	getInvestAmount(int) in parent and child class

4 Overloading with var-args and Wrapper classes

Using var-args

```
class Printer {  
    public void print(String... messages) {  
        for (String msg : messages) {  
            System.out.println(msg);  
        }  
    }  
}
```

Using Wrapper classes

```
class Adder {  
    public void add(int a) {  
        System.out.println("int: " + a);  
    }  
    public void add(Integer a) {  
        System.out.println("Integer: " + a);  
    }  
}
```

Var-args and Wrapper classes can also be used for method overloading.

5 Best Practices

- Use meaningful method names.
- Use overloading for convenience and readability.
- Use @Override annotation to avoid mistakes while overriding.
- Do not reduce the visibility of an overridden method.
- Prefer composition if inheritance is not required.

Same name,
Different power! 🤖

Overloading gives you flexibility. Overriding gives you extensibility. Both make your code powerful!

One Interface
Many Implementations
One Reference
Different Behavior!

Polymorphism in Java

Same Reference, Different Behavior

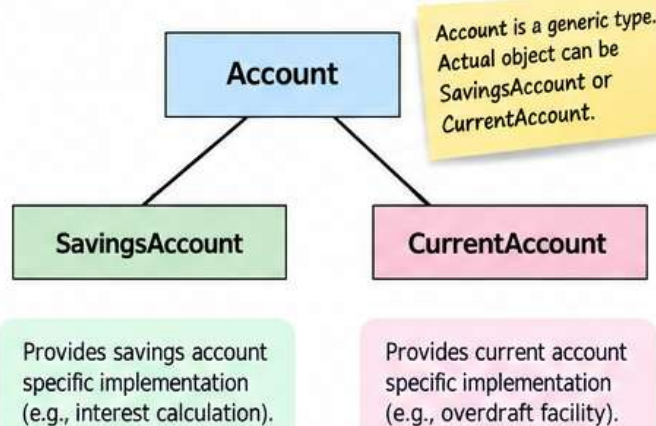
Write Generic Code
Let Objects Decide
at Runtime!

1 What is Polymorphism?

- Polymorphism = many forms of one thing.
- The ability of a single reference of a given type to refer to objects of different types.
- The actual method to be called is decided at runtime based on the object, not the reference type.
- Achieved using method overriding.
- Helps in writing flexible and extensible code.

*"Same reference, different behavior
— decided at runtime!"*

2 Account Example Diagram



3 Code with Method Overriding

```
class Account {  
    public void show() {  
        System.out.println("Account details");  
    }  
}  
  
class SavingsAccount extends Account {  
    @Override  
    public void show() {  
        System.out.println("Savings Account details");  
    }  
}  
  
class CurrentAccount extends Account {  
    @Override  
    public void show() {  
        System.out.println("Current Account details");  
    }  
}
```

Same method name and signature in all classes (method overriding).

4 Polymorphism in Action

```
public class BankApp {  
    public static void main(String[] args) {  
        Account acc; // reference of base class  
  
        acc = new SavingsAccount();  
        acc.show(); // Calls SavingsAccount's show()  
  
        acc = new CurrentAccount();  
        acc.show(); // Calls CurrentAccount's show()  
    }  
}
```

Same reference (Account) is used to refer to different objects.

Output:

Savings Account details
Current Account details

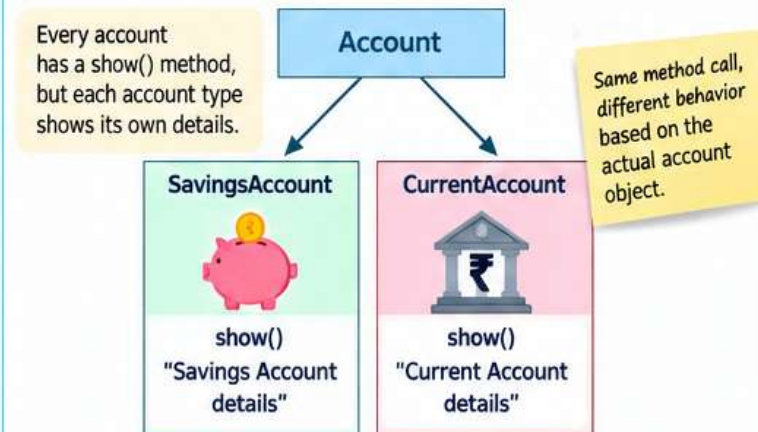
The method to be called is decided at runtime based on the actual object.

5 Key Points

- ✓ Polymorphism allows one reference to refer to different types of objects.
- ✓ Method overriding enables runtime polymorphism.
- ✓ Same method name and signature in parent and child class.
- ✓ Which method to call is decided at runtime, not at compile time.
- ✓ Supports code extensibility and maintainability.
- ✓ Widely used in frameworks (e.g., Spring, Collections).

*"Program to the parent type,
let the object decide the behavior."*

6 Real-world Analogy



Polymorphism: Write once, work with many!

Flexible Code
Future Ready! 😊

Abstract Class in Java

Common Functionality + Incomplete Functionality

Can't create object
Force implementation
Enable code reuse
Useful for generic types! 😊

1 What is an Abstract Class?

- If a class has at least one abstract method it should be declared as abstract class.
- Some time if we want to stop a programmer to create object of some class...
- Class has some default functionality that can be used as it is.
- Can extend only one abstract class.

"Abstract class is a base class with some implemented methods and some abstract (incomplete) methods."

2 Need of Abstract Class?

- sound() method of Animal class don't make any sense.
- ...i.e. it don't have semantically valid definition.
- Method sound() in Animal class should be abstract means incomplete.
- Using abstract method, derivatives of Animal class force to provide meaningful sound() method.

```
class Animal
{
    public void sound()
    {
        System.out.println("Don't know how generic animal sound....");
    }
}
```

3 Correct Declaration

```
class Foo{
    public abstract void foo();
}
```



```
abstract class Foo{
    public abstract void foo();
}
```



```
abstract class Foo{
}
```



4 Use Cases

- Want to have some default functionality from base class and class have some abstract functionality that can't be define at that moment.
- Don't want to allow a programmer to create object of a class as it is too generic.

"Use abstract class when you want common code + incomplete methods and also want to prevent direct object creation."

5 Example: Abstract Class Account

```
public abstract class Account {

    public void deposit(double amount) {
        System.out.println("depositing " + amount);
    }

    public void withdraw(double amount) {
        System.out.println("withdrawing " + amount);
    }

    public abstract double calculateInterest(double amount);
}
```

Abstract method has no body. Implemented methods can be used as it is.

6 Child Classes: Implementation

```
public class SavingsAccount
    extends Account {
    public double calculateInterest(
        double amount) {
        // calculate interest for
        // SavingsAccount
        return amount * 0.03;
    }

    public void deposit(double amount){
        super.deposit(amount); // get code reuse
        // do something else
    }

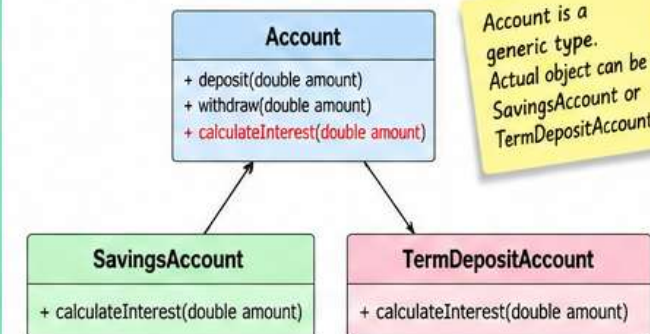
    public void withdraw(double amount){
        super.withdraw(amount); // get code reuse
        // do something else
    }
}
```

```
public class TermDepositAccount
    extends Account {
    public double calculateInterest(
        double amount) {
        // calculate interest for
        // TermDepositAccount
        return amount * 0.05;
    }

    public void deposit(double amount){
        super.deposit(amount); // get code reuse
        // do something else
    }

    public void withdraw(double amount){
        super.withdraw(amount); // get code reuse
        // do something else
    }
}
```

7 Class Diagram (Account)



Account is a generic type. Actual object can be SavingsAccount or TermDepositAccount.

Note: calculateInterest() is abstract in Account and implemented in each child class.

Abstract Class: Define common behavior, leave the specific details to subclasses!

Better Design
Reusability
Maintainable Code 😊

final keyword in Java

Something that can not be changed. | Safer Code | Clearer Intent | Better Design

Same keyword
Many uses
More reliable code!

1 What is final?

- final means something that **can not be change!!!**
- It can be used with variables, method parameters, methods and classes.
- Helps in writing safe, maintainable and clear code.

"Use final when you don't want a value, method or class to be changed."

2 Final Variable

- Becomes **constant**, once assigned then can't be changed.
- Can be instance variable, static variable or local variable.
- Blank final variable must be initialized inside constructor (if instance) or static block (if static).

```
class Account {  
    final int id; // instance final variable  
    static final double RATE = 0.05; // static final  
  
    Account(int id) {  
        this.id = id; // assign once  
    }  
}
```

3 Final Method Arguments (Local final variable)

- Final method arguments can't be changed inside the method.

```
public void getInvestAmount(final int rate) {  
    // rate = 20; // X Compile error  
    System.out.println("Rate: " + rate);  
}
```

Use **final** with method parameters when you don't want the value to be modified.

4 Final Reference vs Object

- A final reference cannot point to another object once assigned.
- But the object itself can be mutable (its state can still change).

```
final Account acc = new Account(101);  
// acc = new Account(102); // X Compile error  
acc.deposit(500); // ✓ allowed  
acc.withdraw(200); // ✓ allowed
```



5 Final Method

- A final method can't be overridden.
- A class containing final method can be extended.
- Killing substitutability (for that method).

```
class Foo{  
    public final void foo(){  
        System.out.println("i am the best");  
        System.out.println("You can use me but can't override me");  
    }  
}  
  
class Bar extends Foo{  
    @Override  
    public final void foo(){ // X Compile error  
    }  
}
```

Final method will be inherited, but child class cannot override it.

6 Final Class

- A final class can't be subclass i.e. can't be extended.
- No method of this class can be overridden.
- Example: String class in Java.
- We can still create object of a final class.

```
package cart;  
public final class Beverage{  
    public void importantMethod(){  
        Sysout("hi");  
    }  
}  
  
package examStuff;  
import cart.*;  
class Tea extends Beverage{ // X Compile error  
}
```

Final class can be instantiated, but can't be inherited.
Example: String class in Java.

7 Account Example Using final

```
public abstract class Account {  
    public void deposit(double amount) {  
        System.out.println("depositing " + amount);  
    }  
  
    public void withdraw(double amount) {  
        System.out.println("withdrawing " + amount);  
    }  
  
    public abstract double calculateInterest(double amount);  
}  
  
public class SavingsAccount extends Account {  
    public double calculateInterest(double amount) {  
        return amount * 0.03;  
    }  
  
    public void deposit(double amount){  
        super.deposit(amount); // get code reuse  
    }  
  
    public void withdraw(double amount){  
        super.withdraw(amount); // get code reuse  
    }  
}
```

8 Comparison Table

Feature	Final Variable	Final Method	Final Class
Meaning	Value/assignment cannot be changed	Cannot be overridden	Cannot be extended
Where used	Local, instance, static variable, method argument	In a class	Class declaration
Inheritance	No impact	Inherited but cannot override	Cannot be inherited
Object creation	Yes	Yes (class can be extended)	Yes (final class can be instantiated)
Purpose	Constant value	Prevent method change	Prevent subclassing

9 Blank Final Variable

- A **final variable** without an initial value is called a blank final variable.
- Must be initialized inside constructor (instance) or static block (static).

10 What final Does NOT Mean

- It does NOT make an object immutable.
- It does NOT prevent a class from being instantiated (final class can have objects).
- It does NOT prevent a method from being inherited (final methods are inherited).

11 Common Use Cases

- Constants (e.g., static final).
- Method arguments (prevent accidental change).
- Prevent overriding of critical methods.
- Designing utility or core classes (final class).
- Used widely in Java libraries (e.g., String class).

12 Java 21 Note

- The **final** keyword works in the same way in Java 21 for variables, methods, method parameters and classes as described here.

final: Fix the important things, keep your code safe and predictable.

Small keyword,
Big impact! 😊

Contract between
Two Parties!

Interface in Java

Define What to Do, Not How to Do!

Multiple classes,
Same contract,
Different behavior!

1 What is an Interface?

- An interface is a contract between two parties.
- It contains only **method declarations** (no method body).
- It can have variables (implicitly **public static final**).
- It is used to achieve **abstraction**, multiple inheritance (in Java), and solve the **diamond problem**.
- A class that implements an interface must provide the implementation of all its methods.

"Interface says **WHAT** to do,
implementation classes decide **HOW** to do."

2 Syntax of an Interface

```
// Interface declaration
interface InterfaceName {
    // variables (implicitly public static final)
    type CONSTANT = value;

    // method declarations (implicitly public abstract)
    returnType methodName(parameters);
    returnType methodName2(parameters);
    // ...
}
```

Key Points:

- Use 'interface' keyword.
- Only method signatures (no body).
- Variables are implicitly public static final.
- Methods are implicitly public abstract.

3 How Compiler Treats It?

```
// What you write
interface Bouncable {
    int i = 9;
    void bounce();
    void setBounceFactor(int f);
}
```



```
// What compiler considers
interface Bouncable {
    public static final int i = 9;
    public abstract void bounce();
    public abstract void setBounceFactor(int f);
}
```

4 Implementing an Interface

```
interface Bouncable {
    void bounce();
    void setBounceFactor(int f);
}
```

```
class Tyre implements Bouncable {
    public void bounce() {
        System.out.println("Bouncing...");
    }

    public void setBounceFactor(int f) {
        System.out.println("Factor: " + f);
    }
}
```

All interface methods must be implemented,
otherwise the class must be declared **abstract**.

5 Rules of Interface (Syntax + Behavior)

1	Declare	Use 'interface' keyword.
2	Methods	Only method declarations (no body). Implicitly public abstract.
3	Variables	Implicitly public static final. Must be initialized at declaration.
4	Access Modifiers	Methods are implicitly public. We cannot use private , protected or default .
5	Instances	Cannot create object of an interface.
6	Implementation	A class must implement all methods. Use 'implements' keyword.
7	Multiple Inheritance	A class can implement multiple interfaces (using comma).
8	Interface Inheritance	An interface can extend multiple interfaces.
9	Constructors	Not allowed in interface.
10	No main()	main() method is not allowed in an interface.

6 More Examples

```
// Multiple interfaces
interface A {
    void showA();
}
interface B {
    void showB();
}
class MyClass implements A, B {
    public void showA() { System.out.println("A"); }
    public void showB() { System.out.println("B"); }
}
```

```
// Interface extending another interface
interface A {
    void showA();
}
interface B extends A {
    void showB();
}
class MyClass implements B {
    public void showA() { System.out.println("A"); }
    public void showB() { System.out.println("B"); }
}
```

7 Key Points to Remember

- ✓ Interface = contract.
- ✓ Only method declarations.
- ✓ No method body.
- ✓ Variables are public static final.
- ✓ Cannot instantiate interface.
- ✓ Class must implement all methods.
- ✓ Use implements keyword.
- ✓ Can implement multiple interfaces.
- ✓ Interface can extend multiple interfaces.
- ✓ Cannot have constructors.
- ✓ No main() in interface.
- ✓ Helps achieve abstraction and solve diamond problem.
- ✓ Supports callback in Java.

8 Common Interview Questions

- 1 What is an interface in Java?
- 2 Why do we need an interface?
- 3 What are the rules of an interface?
- 4 Can we create an object of an interface?
- 5 Can an interface have method body?
- 6 What are the allowed modifiers in an interface?
- 7 Why interface variables are public static final by default?
- 8 Can a class implement multiple interfaces?
- 9 Can an interface extend another interface?
- 10 What is the difference between abstract class and interface?
- 11 How does an interface help in achieving abstraction?
- 12 What is the use of interface in callback mechanism?

Interface: A simple idea, a powerful concept!

Define the contract,
Implement the behavior! 😊

Strings are everywhere
Handle them wisely!

String Class in Java

Immutable | Powerful | Widely Used | Understand Deeply

Same content
Different object!
That's String Pool :) 😊

1 What is String?

- String is a class in java (java.lang.String).
- Represents a sequence of characters.
- Immutable i.e. once assigned then can't be changed.
- Only class in Java for which object can be created with or without using new operator.
- Strings are heavily used in almost every Java application.

"String is immutable, but very powerful!"

2 String Object Creation

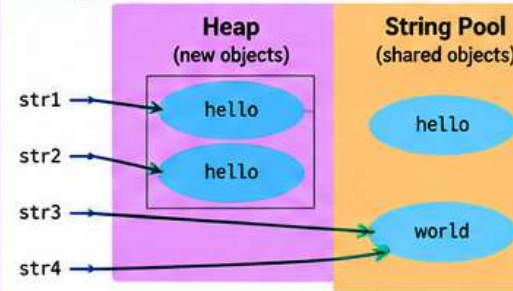
```
String s = "india";           // from String pool
String s1 = new String("india"); // new object
                                // in heap
String s2 = "india";         // from String pool
```

Statement	Where object is created	Notes
String s = "india"	String Pool (in Heap)	Reuses existing object if available.
String s1 = new String("india")	Heap (new object)	Always creates a new object, even if same value exists in pool.
String s2 = "india"	String Pool (in Heap)	Points to existing "india" object (reuses).

3 Inside the JVM (String Pool)

```
String str1 = new String("hello");
String str2 = new String("hello");
String str3 = "world";
String str4 = "world";
```

new always creates a new object. Literal uses pool.



4 Immutability

- Once a String object is created, its value cannot be changed.
- Any operation (like concatenation, replace, substring etc.) creates a new String object.
- The original object remains unchanged (reusable if from pool).

```
String s = "paki";
String s1 = s + "stan"; // creates new object
System.out.println(s);  // paki
System.out.println(s1); // pakistan
```

String s still holds "paki".
Concatenation creates a new object.

5 String Concatenation

- Can be done in two ways:
- Using + operator (operator overloading)
- Using concat() method

```
String s1 = "paki";
String s2 = s1 + "stan"; // using +
String s3 = s1.concat("stan"); // using concat()
```

Both ways create a new String object.
Original string is unchanged.

6 String Comparison

- Never use == to compare String content.
- == compares references (memory address).
- Always use equals() for content comparison.
- Use equalsIgnoreCase() to ignore case.

```
String s1 = "india";
String s2 = "india";
String s3 = new String("india");

System.out.println(s1 == s2); // true (same ref)
System.out.println(s1 == s3); // false
System.out.println(s1.equals(s3)); // true
System.out.println(s1.equalsIgnoreCase("INDIA")); // true
```

7 Common String Methods

Method	Description
charAt(int)	Returns the character at specified index
concat(String)	Appends one string to another
equals(Object)	Checks content equality
equalsIgnoreCase()	Checks equality ignoring case
length()	Returns number of characters
replace(char, char)	Replaces a character with new one
substring(int, int)	Returns a part of a string
toLowerCase()	Converts to lowercase
toUpperCase()	Converts to uppercase
toString()	Returns string representation
trim()	Removes whitespace from both ends

8 Special Points

- String variables in interface are implicitly public static final (used for global constants).
- String is final class (cannot be extended).
- Strings are stored in a special area called String Constant Pool in heap.
- String objects can be created with or without new operator (only class in Java).
- Because String is immutable, it is thread-safe.
- Strings are widely used in collections, I/O, networking, databases, frameworks etc.

"String is final, immutable and cached (via String Pool) — that's why tis so special!"

9 Interview Questions

- What is String in Java?
- Why String is immutable?
- Difference between String s="java"; and new String("java")?
- Why should we use equals() instead of == ?
- What is String Constant Pool?
- How many String objects are created in: String s1="abc"; String s2="abc";?
- What happens when we do s = s + "x" repeatedly in a loop?
- Which are commonly used String methods?

10 Output Example

```
public class StringDemo {
    public static void main(String[] args) {
        String s = "paki";
        String s1 = s + "stan";
        String s2 = s.concat("stan");

        System.out.println(s); // paki
        System.out.println(s1); // pakistan
        System.out.println(s2); // pakistan
    }
}
```

Output:

paki
pakistan
pakistan

Original string
remains unchanged.
New objects are created
for s1 and s2.

11 Key Takeaways

- ✓ String is a class, final and immutable.
- ✓ Can be created with or without new.
- ✓ Use equals() for content comparison.
- ✓ + and concat() both create new objects.
- ✓ Stored in String Constant Pool.
- ✓ Many useful methods are available.
- ✓ Understand String behaviour for better memory usage and performance.

Same value
Different object
Think like JVM!! 😊

Strings may look simple, but understanding them deeply makes you a better Java developer!